

Signalfit SHL Recommendation Agent - Approach

Objective and Design Choices

The goal was to build a conversational SHL assessment recommender that can ask clarifying questions, retrieve catalog-grounded products, and preserve a working shortlist across turns. I separated the system into a 'backend/' FastAPI service and a static 'frontend/' UI. The backend is stateless: every '/chat' request sends the full message history, and the agent reconstructs the current state from that history. This keeps deployment simple and avoids server-side conversation storage.

The backend pipeline has three stages. First, an LLM extracts structured state: operation ('clarify', 'recommend', 'refine', 'compare'), role, seniority, domain, test types, requirements, purpose, and clarification intent. Second, retrieval builds a bounded candidate pool from the SHL catalog. Third, a reranker LLM selects only from retrieved/prior candidates and returns a concise reply plus catalog names/URLs. Prior shortlist reconstruction is important for refinement turns: the latest assistant-emitted 'Shortlist:' is treated as the working state, so removals and confirmations do not silently resurrect old items.

Retrieval Setup

The catalog is stored in 'backend/data/shl_catalog.json', with a FAISS index and cached embeddings. Retrieval combines lexical scoring, metadata filters, and embedding search. The query is built from extracted state: role, seniority, requirements, domain terms, hard test types, and optional soft test types. Metadata boosts reward matching job levels and SHL test categories. Rule boosts handle common domain signals such as leadership, technical skills, selection, development, remote/adaptive constraints, and report/artifact penalties.

I avoided making personality or aptitude mandatory defaults. Instead, I added 'soft_test_types', currently limited to 'A' and 'P'. The state extractor may mark aptitude/personality as optional when they could complement the role, but retrieval only applies a very small boost and does not treat them as hard filters. The reranker is explicitly told that soft types are optional and should not crowd out direct role-critical assessments.

Prompt Design

There are two main prompts. The state prompt forces a compact JSON schema and decides whether the next turn should clarify or recommend. A key improvement was teaching it to merge short clarification answers, such as "US", "English", "backend", or "selection", into the existing state instead of replacing the whole query. This addressed multi-turn failures where the system lost the original role context after the user answered a narrow clarification.

The reranker prompt is deliberately restrictive. It can only select from provided candidates, must preserve exact catalog names, must not invent URLs, and must not pad to 10 recommendations. For refinements, the previous shortlist is the working shortlist; previous items are preserved unless the latest user message or updated requirements justify removal. If the model omits a prior item, it must return a grounded omission reason.

Evaluation Method

Evaluation is implemented in 'backend/evaluation/eval.py'. It parses the public sample conversations into recommendation checkpoints. For each checkpoint, expected catalog names are extracted from the trace tables and canonicalized by URL/name against the SHL catalog. The agent response is scored with Recall@10: expected products recovered in the returned recommendation list divided by expected products. The script also validates the response schema and reports mean Recall@10 overall and by conversation.

I used both full eval runs and targeted probes. Targeted probes were useful when full LLM-backed runs were slow or unstable: for example, I inspected C1 turn 3 state/response directly and tested whether retrieval could surface the expected candidates before reranking. I also checked degraded-state scenarios for C3 to isolate whether zero recall came from retrieval or from state reconstruction.

What Did Not Work

The first attempt to stop early recommendations used a deterministic controller-level screening gate. That was too blunt. It blocked valid recommendation checkpoints, reduced mean recall, and interfered with retrieval. I removed that approach. The better fix was to keep retrieval unblocked and make clarification a state-level decision: if the extractor has a live unanswered clarification, ask it; if the clarification has already been answered in the structured state, clear it and recommend.

Another issue was stale clarification intent. In C1 turn 3, the user answered the purpose question with

selection, but state still contained the previous clarification question. The agent repeated the question and got 0 recall. I added normalization that clears an answered clarification, improving that checkpoint from 0.000 to 0.333 before further retrieval tuning.

C3 showed a different failure mode. The expected stack was a spoken language screen, call simulation, customer-service fit, and phone simulation. Retrieval could find all four when the state preserved the contact-center context, but collapsed when the latest short answer "US" replaced the full state. The generic prompt fix for short clarification answers addressed this without hardcoding C3 products.

Measuring Improvement and Tool Use

Improvement was measured with the same Recall@10 eval checkpoints, schema validation, and targeted before/after probes. I tracked whether fixes improved the failing checkpoint without blocking retrieval or overfitting to one trace. I also used local checks such as 'py_compile', retrieval top-k inspection, and eval dry-runs after restructuring.

I used OpenAI Codex as an agentic coding assistant for codebase navigation, refactoring, prompt iteration, targeted debugging, and generating this report. I did not use no-code builders. All recommendation logic remains code/prompt based and catalog-grounded.